FOUNDATIONS OF DEEP LEARNING

What is Deep Learning?

Imagine teaching a machine the way you would teach a child. You show a child many pictures of cats. At first, the child cannot recognise one. But after seeing enough examples, the child starts noticing patterns pointed ears, whiskers, soft fur, bright eyes. Slowly, recognition becomes automatic.

Deep Learning works in a very similar way.

Deep Learning is a branch of Artificial Intelligence that allows computers to learn patterns directly from data using structures called Artificial Neural Networks. Instead of programming explicit rules like:

"If it has whiskers and pointed ears, then it is a cat," we give the machine thousands (or millions) of examples and allow it to learn those rules by itself.

It is called *deep* because the learning happens through multiple layers of processing each layer learning something slightly more complex than the previous one.

Neural Networks

A **neural network** draws its conceptual inspiration from how the **human brain** processes information through interconnected **neurons**.

In the brain, neurons communicate through electrical impulses; in artificial networks, neurons communicate through **mathematical signals** and **weighted connections**.

A **neuron** in neural network is a mathematical unit that processes inputs using weights, bias, and an activation function to produce an output. While a single neuron handles simple linear decisions, multiple layered neurons can learn complex patterns, such as identifying objects in images.

Biological Neuron: Structure

Part	Function

Dendrites	Receive input signals from other neurons
Cell body (Soma)	Processes the incoming signals and decides whether to activate
Axon	Transmits output signals to other neurons
Synapse	Connection point between neurons; the signal strength can increase or decrease with learning

Mathematically, a neural network is a function $f(X, W)$ that maps input features X to an output Y through a series of **weighted transformations** and **non-linear activations**.

In simple terms neural networks learn from data by adjusting internal parameters (weights and biases) so that their predictions get closer to actual outcomes.

Think of a neural network as a layered decision process:

- The **input layer** receives raw data.
- The **hidden layers** progressively learn abstract features (such as edges, shapes or word patterns).
- The **output layer** produces final predictions (such as a class label or score).

➢ **Perceptron and MLP**

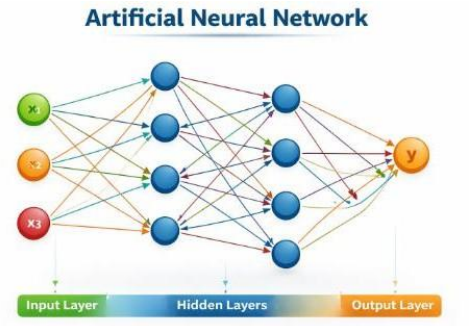
- The Perceptron is the first implementation of an artificial neuron used for binary classification. It adjusts weights based on errors and works only for linearly separable problems.
- Its limitation (cannot solve non-linear problems like XOR) led to the development of the Multi-Layer Perceptron (MLP).
- MLP includes one or more hidden layers, allowing the model to learn non-linear and complex relationships using backpropagation.

Example: Spam detection (Perceptron for simple cases) vs. house price prediction (MLP for complex feature combinations).

➢ **Artificial Neural Networks (ANN)**

- An ANN is a system of interconnected artificial neurons organised into layers (input, hidden and output layer).
- It learns through forward propagation (prediction), loss calculation (error measurement), and backpropagation (weight updates).
- MLP is one common type of ANN, and deeper ANNs form the base of modern deep learning systems.

Example: Face recognition, speech recognition, recommendation systems.



A neural network is a computational model inspired by the human brain that learns and processes information. It consists of interconnected layers of neurons that work together to identify complex patterns in data.

Mathematically, a neural network is a function $f(X, W)$ that maps input features X to an output Y through a series of **weighted transformations** and **non-linear activations**.

In simple terms neural networks learn from data by adjusting internal parameters (weights and biases) so that their predictions get closer to actual outcomes.

Think of a neural network as a layered decision process:

- The **input layer** receives raw data.
- The **hidden layers** progressively learn abstract features (such as edges, shapes or word patterns).
- The **output layer** produces final predictions (such as a class label or score).

Core Components

Component	Description	Example
Neuron (Node)	The basic computational unit. Each neuron performs a weighted sum of inputs, followed by an activation function	$z = \sum x_i w_i + b$
Weights (W)	Numerical parameters that determine the importance of each input feature	High weight → Stronger influence
Bias (b)	A constant that allows the activation threshold to shift	Helps fit data not passing through the origin
Activation function (f)	Introduces non-linearity, allowing networks to model complex relationships	ReLU, Sigmoid and Tanh
Layers	Arrangements of neurons; includes input, hidden and output layers	Multilayer feedforward design

Working of a Single Neuron

A neural network starts with **inputs**, which are the features or data given to the model. These inputs can be numbers like study hours, attendance, image pixel values, or any measurable information related to the problem. Each input is fed into a neuron with an associated weight that represents its importance.

Inside a single neuron, the inputs are multiplied by their respective weights and summed together. A bias term is then added to adjust the result. This combined value is passed through an activation function to produce the final output. Mathematically:

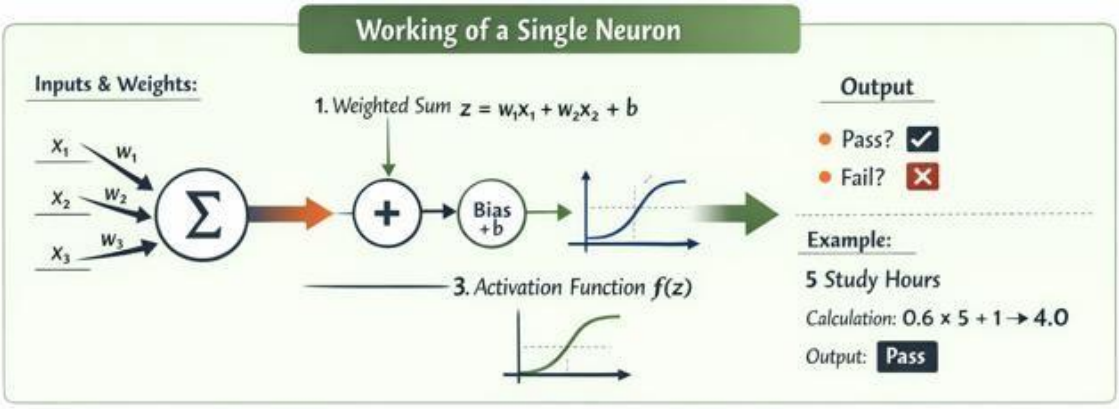
$$z = w_1 x_1 + w_2 x_2 + \ldots + b$$
$$a = f(z)$$

Here, x represents inputs, w represents weights, b is bias, a is output and $f(z)$is the activation function.

The **output** is the final prediction generated after this computation. Depending on the problem, the output may be:

- A binary value (Pass/Fail)

- A category (Cat/Dog/Car)
- A continuous number (House price)



Activation Functions

In a neural network, each neuron calculates a weighted sum of inputs and adds a bias. However, this value alone is not enough to make meaningful decisions. This is where the **activation function** comes into play.

An activation function is a mathematical function applied to the output of a neuron. It determines whether the neuron should be activated (send signal forward) or not.

Types of Activation Functions

Activation functions can be broadly classified into:

1. **Linear Activation Function:** Output is directly proportional to input; does not introduce non-linearity.
    - $f(x) = x$
2. **Non-Linear Activation Functions:** Introduce non-linearity, allowing the network to learn complex patterns and relationships.
    - **Sigmoid:** Squashes input to a range between 0 and 1; useful for probabilities.
        - $\sigma(x) = \frac{1}{1+e^{-x}}$

- **Tanh (Hyperbolic Tangent):** Squashes input to a range between -1 and 1; zero-centered.
  - $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

- **ReLU (Rectified Linear Unit):** Outputs zero for negative input and linear for positive; widely used in hidden layers.
  - $\text{ReLU}(x) = \max(0, x)$

- **Leaky ReLU:** Like ReLU but allows a small slope for negative inputs to avoid dying neurons.
  - $\text{Leaky ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$

- **Softmax:** Converts a vector of values into probabilities that sum to 1; used in output layer for classification.
  - $\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$

Parameters and Hyperparameters

- **Parameters** are the elements the network **learns during training**.
  - **Weights (w):** Determine how strongly each input influences a neuron.
  - **Biases (b):** Allow neurons to shift the activation function and fit the data better.
- **Hyperparameters** are **set before training** and guide the learning process.
  - Examples include: learning rate, number of layers, number of neurons per layer, activation functions, batch size, and number of training epochs.

Think of parameters as the network's "knowledge" and hyperparameters as the "rules of learning."

Forward Pass in FCFNNs

In a Fully connected feedforward neural network (FCFNN), information flows strictly forward from the input layer to the output layer, layer by layer. This process, called the forward pass, transforms raw inputs into meaningful predictions while computing the activations of each neuron.

1. **Input Layer:** Receives raw features from the dataset and passes them to the first hidden layer.

2. **Hidden Layers:** Each neuron computes a weighted sum of its inputs, adds a bias, and applies an activation function:

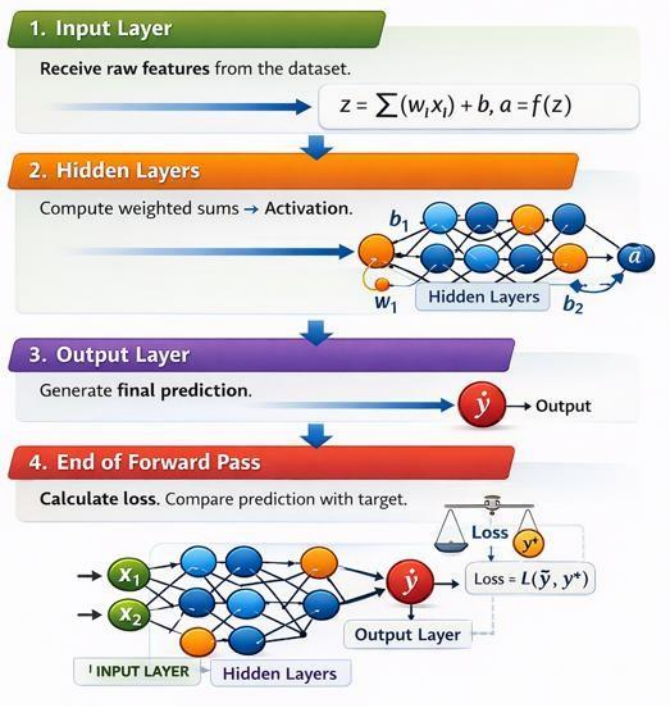
$$z = \sum(w_i x_i) + b, a = f(z)$$

The activation is then passed to the next layer.

3. **Output Layer:** Computes the final prediction using an appropriate activation function (e.g., softmax for classification or linear for regression).

4. **End of Forward Pass:** The predicted output is compared with the target using a loss function. This error is then used later during backpropagation to update the network parameters.

The forward pass is performed using the **feedforward algorithm**, which is the systematic procedure that defines how inputs are processed to generate the network's output as illustrated below.



Loss Functions

A **loss function** is a measure of how well a neural network's predictions match the actual target values. It tells the network **"how wrong it is"** and provides guidance for improving predictions. Without a loss function, the network wouldn't know in which direction to adjust its parameters.

Key Points:

1. **Purpose:** Quantify the difference between predicted outputs and actual targets.
2. **Example for Regression:** Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

3. **Example for Classification:** Cross-Entropy Loss

$$\text{Loss} = -\sum_{i} y_i \log(\hat{y}_i)$$

4. **Effect:** The higher the loss, the worse the network's predictions; the goal of learning is to **minimise this loss** over the training data.

The **loss function** acts like a compass during training: it points the network in the right direction for adjusting weights and biases.

Neural Network Learning

Learning in a neural network is the process of adjusting **parameters** (weights and biases) to reduce the loss. This happens in multiple steps:

1. **Forward Pass:** The network computes predictions for the input data, and the loss function measures how far the predictions are from the actual targets.
2. **Backward Pass (Backpropagation):** The network calculates gradients of the loss with respect to each parameter using the chain rule. This tells us **how much each weight and bias contributed to the error**.

3. **Parameter Update:** Using an optimisation algorithm like **Gradient Descent**, the network updates its parameters in the direction that reduces the loss:

$$w := w - \eta \frac{\partial L}{\partial w}, b := b - \eta \frac{\partial L}{\partial b}$$

Where

- $w \rightarrow$ The weight of a connection in the network (a learnable parameter).
- $b \rightarrow$ The bias of a neuron (another learnable parameter).
- $L \rightarrow$ The loss function, which measures how far the network's prediction is from the true target.
- $\frac{\partial L}{\partial w} \rightarrow$ The **gradient of the loss with respect to weight** w. It tells us how much changing this weight will change the loss.
- $\frac{\partial L}{\partial b} \rightarrow$ The gradient of the loss with respect to the bias b.
- η(eta) $\rightarrow$ The **learning rate**, a small positive number that controls **how big the step** is when updating parameters.

Reducing Loss

- **Gradient ($\partial L / \partial w$)**: Shows the direction and magnitude in which the loss increases with respect to that weight.
- **Minus sign (-)**: We subtract the gradient because we want to **move in the direction that decreases the loss**, not increases it.
- **Learning rate (η)**: Scales the step size so we don't overshoot the minimum

Learning is Iterative

- The network repeats this process over many **epochs** (full passes over the training dataset).
- With each iteration, the **loss decreases**, and the network's predictions become more accurate.

- Proper choice of **loss function and learning rate** is critical for stable and effective learning.